

# Assignment 1: MapReduce (10 pt)

Course: Distributed Systems

**Deadline: September 29, 2026, at 23:59 CEST.**

Create a group of two before starting. One partner creates the private template repository and gives the other **Write** access. Record both students here.

| student name | GitHub handle |
|--------------|---------------|
| TODO         | TODO          |
| TODO         | TODO          |

In this 2-week assignment, you will learn the basics of distributed computing by implementing MapReduce. You will also gain hands-on experience with gRPC, an open-source and high-performance framework for Remote Procedure Call (RPC) including simulating larger distributed systems, and with Git, a distributed version control system.

**The assignment repository:** <https://github.com/HSG-DS-HS26/BCS-DS-Assignment-1>

One group member should create a repo by clicking **Use this template** to create the group's private repository. Do not fork the repository or clone it directly! This just the template. The template features allows us to have your repository private instead of public. See [SETUP.md](#) for a more detailed walkthrough.

|                         |                                                       |          |
|-------------------------|-------------------------------------------------------|----------|
| <b>Before you start</b> | one shared private copy of the assignment repository  | <b>0</b> |
| <b>Task 1</b>           | identify characteristics of a distributed system      | <b>1</b> |
| <b>Task 2</b>           | MapReduce on one machine                              | <b>2</b> |
| <b>Task 3</b>           | MapReduce on a cluster                                | <b>4</b> |
|                         | <b>commit and push</b>                                |          |
| <b>Task 4</b>           | defend your submission (experimental)                 | <b>1</b> |
| <b>Task 5</b>           | pitch design decisions                                | <b>2</b> |
|                         | <b>commit and push, then submit the PDF on Canvas</b> |          |

Make sure to follow the correct order: Push Task 3 before the viva, because the viva reads what is on GitHub. Pull after the viva, because it pushes its record to your repository. Push Task 5 before you write the Canvas PDF, because it includes the latest published commit.

---

## Before you start (0 pt)

---

Work in one shared **private** repository and give both partners and the TA the required access. [SETUP.md](#) explains the setup.

### To do

- ☐ Create the pair's **private** repository, give your partner Write access, and give the TA Read access. For an archive submission, or if the automated AI viva may not read your repository, tell the TA before the deadline — [SETUP.md](#)

**Work in a Codespace or the Dev Container, not on your own machine.** Every command in this repository is written for the POSIX shell those give you, and everyone then has the same toolchain and the same JDK. On Windows especially: the commands still work as printed inside either of them. If you insist on a Windows host, [the README](#) says what each command becomes in PowerShell.

---

## ★ Task 1: Spotting inner workings of a Distributed System (1 pt)

---

Tundrell Outfitters, a fictional retailer, sells parkas in its shops and online. Staff use one count of each product to decide whether they can accept an order, move stock between shops, or place a supplier order. A wrong count creates real bookkeeping problems, so the company keeps three copies of the data in three data centres.

A year ago Tundrell paid a consulting company to build two versions of this store. The consultants labelled one an AP system and the other a CP system, then left. **The labels were lost.** The implementations use the same cluster and handle a network partition differently. The company has hired you to recover the labels from the systems' behaviour.

You will not be told which is which. Run both, watch what each one does when a network partition occurs, and say which prioritizes consistency and which prioritizes availability. There is no code to write here. Read the run's behaviour and support your answer with evidence from it.

### To do

- ☐ Read `1-spot/README.md`: how to read a run
- ☐ Run `simulations/system-1.yaml` and `simulations/system-2.yaml` from the console, and keep both under **Runs**
- ☐ fill out `1-spot/QUIZ.md`

### Grading

The right label for each system, and evidence that came from a run will get graded.

---

## ★ Task 2: MapReduce on one machine (2 pt)

---

[Google's 2004 MapReduce paper](#) describes a programming model in which you write Map and Reduce functions while a framework handles their execution on a cluster of machines. This task runs those functions in plain Java on one machine, where nothing can fail and nothing has to be sent anywhere (compared to task 3, where this will be implemented as a distributed system).

The starter interfaces are generic so one pipeline can handle different key and value types. If Java interfaces are new to you, read [Oracle's interface tutorial](#). [2-mapreduce/README.md](#) covers the files, the interfaces, the tests, the output format, and optional questions about language signals and investment data.

### To do

- ☐ Read [2-mapreduce/README.md](#)
- ☐ Implement **word count** (see slides from Lecture 1): `Map` and `Reduce`
- ☐ Implement **reverse links** (see slides from Lecture 1): `Map` and `Reduce`
- ☐ Uncomment both jobs in `src/Main.java` and run it
- ☐ `test/Invariants.java` passes — it checks properties, not answers

### Grading

Both jobs producing the required output and don't break the invariants.

---

## ★ Task 3: gRPC tutorial and MapReduce across a cluster (4 pt)

### Tutorial: introduction to gRPC (strongly recommended)

Task 3 uses gRPC for the MapReduce execution model. This tutorial introduces gRPC and protobuf first.

The tutorial builds a **thumbnail service**. It accepts an image, creates a preview, and keeps it until a client requests it. One image is one unit of work, so the console's bytes, time, and cost have a clear unit.

The same handlers run first on one machine, then across four nodes, and then under a node failure and network faults. Your Java Code remains unchanged between those runs; the scenario changes.

Work through [3-distributed/tutorial/README.md](#) in order.

### To do

- ☐ **Stage 1, plain gRPC.** Type the protobuf specification, the server and the client yourself. Run them against a real socket on port 50052 until the client prints its line
- ☐ **Stage 2, adopt it.** `dissaly adopt` on that same project
- ☐ Add one message, make the two source changes, declare the call costs
- ☐ Run all four simulations from the console on [port 19842](#), manual open alongside

### Grading

The tutorial is strongly recommended but not graded. [.solution/plain/](#) and [.solution/dissaly/](#) hold finished versions. Use them when you are stuck, not instead of typing.

**An AI agent may help you here** — let it guide you, ask it questions, ask it to challenge you in the specific understanding with questions and so on.

### Exercise: MapReduce across a cluster

The same word count task as in Task 2 should now be implemented across one Master and seven Workers that share no memory, communicate only through gRPC, and have to survive one of them dying mid-job. **Your task is the communication: what one machine tells another, and the handlers at both ends of it.**

Lecture 2 draws the job as five steps:

|                |    |        |    |                   |    |           |    |                 |
|----------------|----|--------|----|-------------------|----|-----------|----|-----------------|
| 1. Split input | -> | 2. Map | -> | 3. Shuffle & Sort | -> | 4. Reduce | -> | 5. Output       |
| Corpus         |    | yours  |    | Shuffle.java      |    | yours     |    | GrpcUserProgram |

Steps 2 and 4 are the calls the Master makes and the handlers that answer them. Step 3 is supplied, and so are task assignment, the failure policy and the scenarios. [COMPONENTS\\_OF\\_MAPREDUCE.md](#) records what was decided for you and why. You are not examined on those decisions, but Task 5 measures what they cost.

## To do — read

- ☐ `COMPONENTS_OF_MAPREDUCE.md` — the components, what each has to guarantee, and which ones are built
- ☐ `infrastructure.proto` and `infrastructure/Shuffle.java`, which your `Worker` extends and which decides what your reduce handler has to do
- ☐ `mapreduce/RunJob.java`, `infrastructure/JobContext.java` and `infrastructure/Master.java`. `JobContext` contains everything the Master can tell a Worker, which bounds your messages. `Master.java` contains the fixed liveness call

## To do — design

- ☐ `COMPONENTS_OF_MAPREDUCE.md` again, component by component, answering its questions before writing anything
- ☐ then `worker.proto`. The service and rpc names are printed there and are fixed, because the Task 5 scenarios key on them. What each message **carries** is yours
- ☐ name each message and field **in words** before you write any protobuf

## To do — build, in this order

Run the project after each step.

- ☐ **The messages, and the rpcs inside `service Worker`**. The project then builds, and every call fails `UNIMPLEMENTED`
- ☐ **The heartbeat**. `Heartbeat.java` explains its requirements
- ☐ **The map handler** and `RunJob.mapRequest`. Map works while Reduce fails
- ☐ **The reduce handler**, `RunJob.reduceRequest`, and the marked merge of a reduce reply in `RunJob.java`
- ☐ `./gradlew run` prints **24 rows and a total of 20 000**. This smaller plain-gRPC workload provides fast feedback before Task 5 uses 10 000 000 words in DISSALy.

Be aware: The row count is easy to get right by accident, but the total is where you master.

## Graded

The plain gRPC system:

- correct output
- the messages you designed in `worker.proto` and the handlers that serve them in `Worker.java` and `Heartbeat.java`, together with the holes in `RunJob.java`;

Full procedure in `3-distributed/exercise/README.md`.

---

## Hand in Task 3: commit and push

---

When Task 3 runs successfully, **commit and push** to the group repository. The viva in Task 4 reads that repository on GitHub, so work that exists only in a Codespace or laptop is unavailable to it.

Archive submissions require advance notice to the TA. Send the final assignment directory as a `.zip` by the agreed route; its viva is a direct Teams call with the TA. The same Teams viva applies when the automated AI viva may not read your GitHub repository.

### To do

- ☐ Task 3 runs successfully
  - ☐ **Commit and push**, or send the agreed `.zip` archive
  - ☐ Tell the TA before the deadline if you don't want to defend the submission (viva) with the provided tool, but with a Teams call
-

## ★ Task 4: defend your submission (1 pt, required)

---

With Task 3 pushed, open [the viva site](#) and sign in with GitHub. Select your repository and the commit you pushed, then answer the viva's questions about your code.

When the viva ends, **it commits its record to your repository and pushes it itself**. That commit is on GitHub and not yet in your Codespace or on your laptop, so **pull** before you do anything else. If you skip the pull, your next push is rejected, because GitHub has a commit you do not.

Archive submissions and students who withhold GitHub access from the automated AI viva complete the viva directly with the TA on Teams. For this you need to inform the TA beforehand.

### To do

- ☐ Task 3 is committed and pushed
- ☐ Open [the viva site](#), select your repository and commit, and answer its questions
- ☐ **Pull**, and check that `.viva/` is now in your repository. In VS Code: **Source Control** -> **Sync Changes**, or `git pull` in a terminal
- ☐ Attend the agreed direct Teams viva if you use an archive submission or withhold GitHub access from the automated AI viva

### Graded

The actual content of the viva is not yet graded, but you need to have the viva logs in the repository to get the point here.

---



## ★ Task 5: the pitch (2 pt)

---

Tundrell Groupt (the parent company of Tundrell Outfitters) was really happy with your work in task 1. Now they want a new feature: They want to count words across its product catalogue and supplier documents with the gRPC MapReduce system from Task 3.

Since you have already implemented everything, the only thing that is missing is a five-minute pitch to the technical management which focusses on the exact details of configuration and its resilience against stragglers and partial failures.

[5-presentation/TASK5.md](#) explains the task in more detail. See [5-presentation/README.md](#) for technical details.

### To do

- ☐ **Switch Task 3 from plain gRPC to DISSALy.** Commit first because adoption refuses a dirty tree. [TRANSFORMATION.md](#) includes the procedure and the required `Shuffle.java` edits. The lab then appears on [port 19843](#), where the simulator can evaluate Worker failures, stragglers, and configuration cost
- ☐ Run the scenarios in `simulations/`, then the failure overlays against the configurations you are considering
- ☐ Pick three configurations including their overlays and capture the screenshots and film clips
- ☐ Choose one Task 5 media option: commit both `slides.pdf` and `presentation.mp4` to [5-presentation/](#), or upload both files to Canvas with the submission PDF

### Grading

Follow the presentation requirements strictly. The three configurations must differ in M and R, each claim must come from a run you performed, and the deck must end with a justified recommendation.

---

## Submit on Canvas

---

The last step is a PDF on Canvas that names your repository and the commit to grade. A tool in the repository writes it, and it refuses to while GitHub is missing any of your work.

1. **Commit and push** the repository work. Choose either the repository or Canvas option for the Task 5 media files. For the Canvas option, move `slides.pdf` and `presentation.mp4` outside the repository before pressing **Hand in**. If you have not pulled since the viva, pull first.
2. **Write the PDF**. In VS Code, press **Hand in** in the status bar at the bottom, or run **Terminal -> Run Task -> Hand in: write the PDF for Canvas**. From a terminal:

```
./submit.sh
```

**Windows (PowerShell)**. We recommend a Codespace or the Dev Container, where the button and the command above work as printed. On a Windows host:

```
.\submit.ps1
```

If Windows refuses to run scripts, type `java submission\Submission.java` instead: it is the command `submit.ps1` runs.

It writes `<your-username>--<repo>--submission.pdf` in the repository root and lists what it found. If it says **No PDF written**, it also says why and what to do: usually commit, push, or pull.

3. **Upload that PDF** to the assignment on Canvas. With the Canvas media option, upload both presentation files as well. The PDF will mark those files as missing from GitHub.

If you change anything after that, **commit and push again, then write and upload a new PDF**. The PDF names one commit, and that commit is what gets graded.

### To do

- ☐ Everything committed and pushed, and pulled since the viva
  - ☐ **Hand in** pressed, and the PDF lists your repository and the commit you expect
  - ☐ PDF uploaded to Canvas, with both Task 5 media files when using the Canvas media option
-

## AI agents

---

Ask an agent about **the tools**: gRPC, protobuf, Gradle, the simulator, an unrecognised error, the tutorial, or adoption.

Design the **point-carrying work yourself**: Task 2's `Map` and `Reduce` , and Task 3's messages in `worker.proto` , the handlers in `Worker.java` and `Heartbeat.java` , and the holes in `RunJob.java` . Task 4 asks you to defend those decisions.